Ch2: Understanding Foundation Models:-

Training data — Modeling — Post-training — Sampling

Which model to use, and adapting it?

- Training data → reveals a great deal about model's capability.
- Modeling → Transformers.
- Post-training → Align the model with human preferences.
- Sampling → how a model chooses o/p from all possible options. (hallucinations, inconsistencies explained). (boost performance)

1. Training Data

Model → As good as the data it's trained on.

→ Common Crawl (lot of fake news)
→ C4 (cleaned version).
→ Data Quality: (small but high quality, can outperform large models).

→ Multilingual models :-
- → low resource languages. (in Common Crawl)
- → GPT models perform better in English than low resource languages.
- → Under-representation in training data causes this underperformance.
- → Even if we translate to English, we can have information loss during translation.
- → Models are slow & expensive for non-English languages.

A model's inference latency & cost $\propto$ no. of tokens in I/P & response

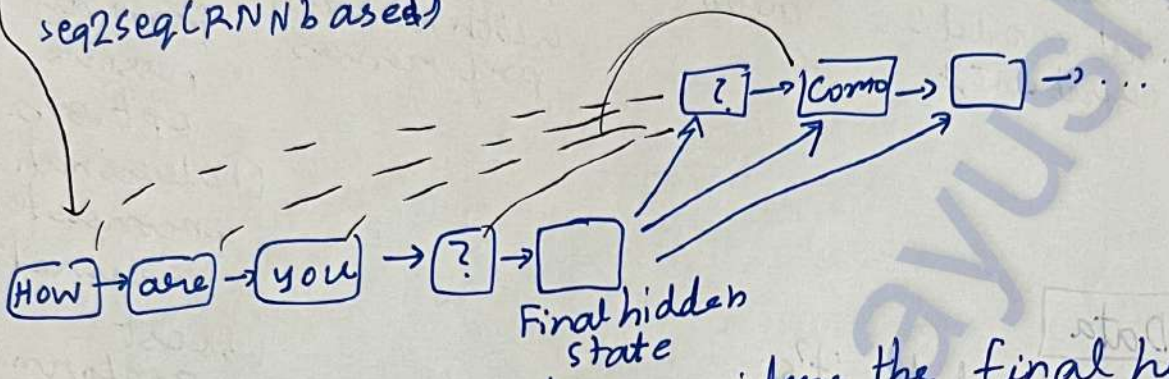
For Eg: Hindi requires more tokens for the same meaning compared to English.

Domain Specific Models :- Domain Specific Training data.
another way is to run benchmarks and see in which domain the model works better.

2. Modelling

Transformer architecture

seq2seq (RNN based)



Problms :→ (i) decoder only considers the final hidden state of I/P
(2) seq. Nature of I/P, slow. Vanishing & exploding gradient problem.

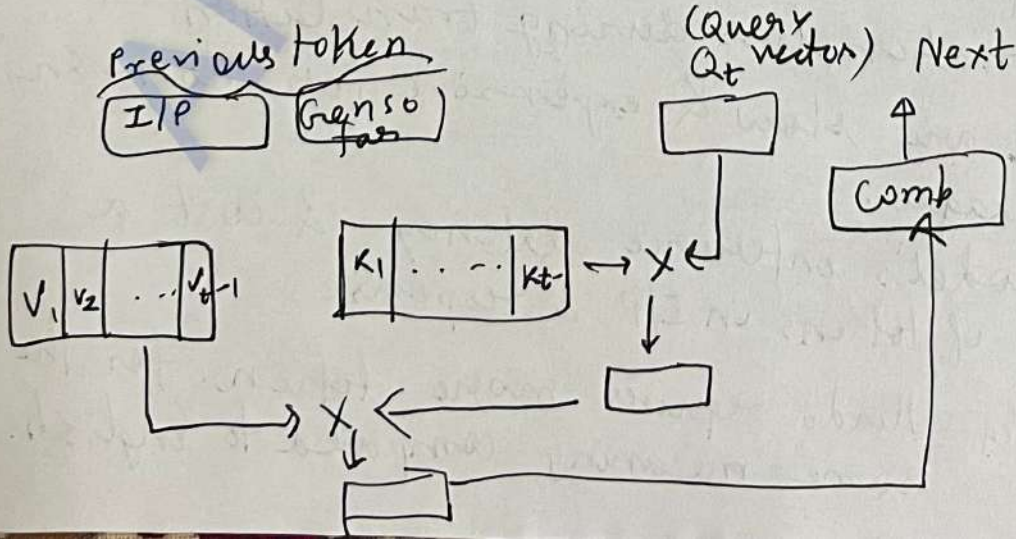
Attention ① weigh the importance of different I/P tokens, when gen. O/P

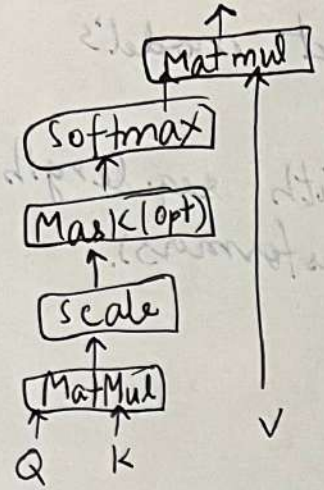
② I/P tokens can be processed in parallel.

Inference for Transf- based LMs.

① Prefill: Model processes I/P tokens in parallel.
(K, V) vectors ∀ I/P tokens

② Decode: The model generates one O/P at a time.
token





x

$K = xW_K$

$V = xW_V$

$Q = xW_Q$

Multi-headed.

Llama 2-7B

4096: hidden

32 Heads

$\frac{4096}{32} = 128$ (vectors are split)

$$\text{Attn} = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Concatenate the O/Ps of attⁿ heads.

O/P projection matrix (transformer).

Attⁿ Module:- W_Q, W_K, W_V, W_O

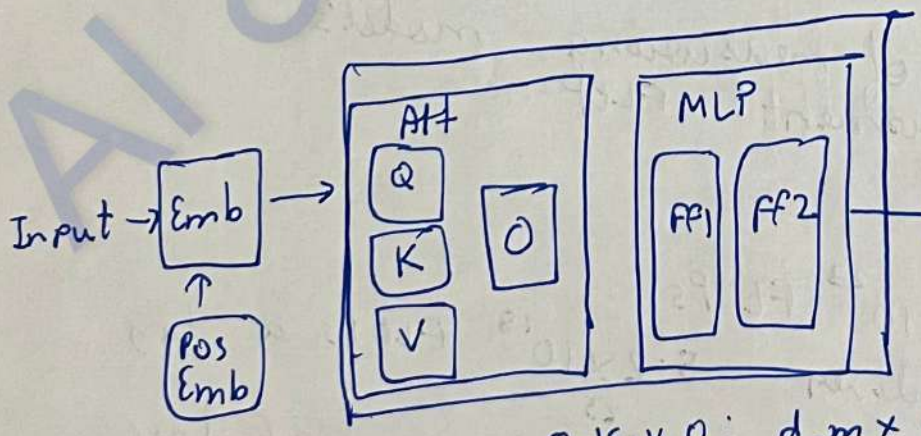
MLP Module: linear layer + Nonlinear activation fⁿs.
(linear transformations)

(ReLU, GeLU)

max(0, x)

An embedding Module before the transformer blocks

An O/P layer after the transformer blocks.
(Model's O/P vectors → into token probabilities).



Q, K, V, O: d_m × d_m

FF1: d_m × d_ff

FF2: d_ff × d_m

Emb: V × d_m

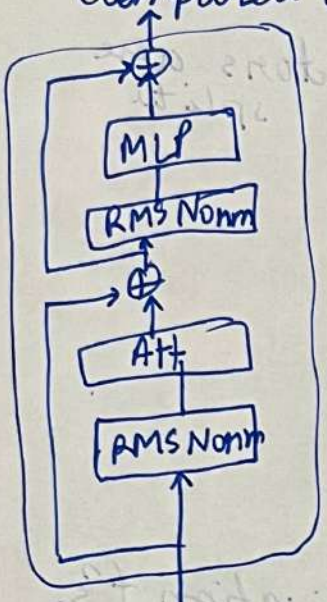
Pos Emb: C × d_m

O/P: d_model × V

AI & ML with Aayush

So, the increased context length doesn't impact model's total no. of params.

Mamba:- Inf. computation scales linearly with seq. length (compared to quadratic scaling for transformers).



Transformer layer

MoE

Mamba

Attention MoE layer.

Mamba layer

Model Size: Llama-13B

For 7B model, 2 bytes

GPU needed during inf: 14GB.

Sparse Models: MoE: (Only a subset of experts remains active to process the token)

No. of tokens: Measurement of the data.

training tokens = No. of tokens the model is trained on.

Standardized way of measuring model's compute requirement: FLOP

256 H100's

GPT-3: 3.14×10^{23} FLOPs

H100: can deliver 5.2×10^{18} FLOPs a day

To train (GPT3-175B) $\frac{3.14 \times 10^{23}}{5.2 \times 10^{18} \times 256} \approx 236$ days

AI & ML with Aayush

70% utilization

$2/hr

1 H100 GPU

GPT-3-175B:

$2/H100/hr × 256 H100 × 24 hr × 236 days / 0.7

≈ $4M.

Model's Scale

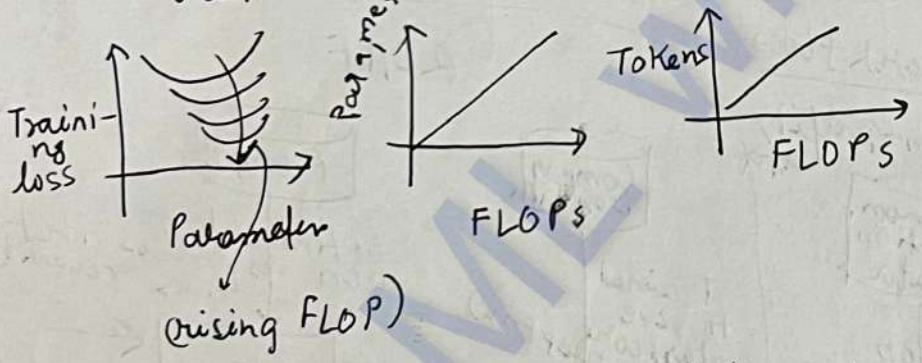
- No. of params (learning capacity)
- No. of tokens a model was trained on (how much it learnt)
- No. of FLOPs, (proxy for training cost)

Scaling laws:-

Building Compute-optimal models:-

Compute-optimal.

$N_{train\ tokens} \approx 20 \times$ model size.



Parameter (using FLOP)

Optimizing model quality given a compute budget.

Sometimes a slightly smaller model for better inference.

Hyperparams:- How the model learns?

#layers,
dim. of hidden layer
Vocab Size.

Batch size,
#epochs
lr

AI & ML with Aayush

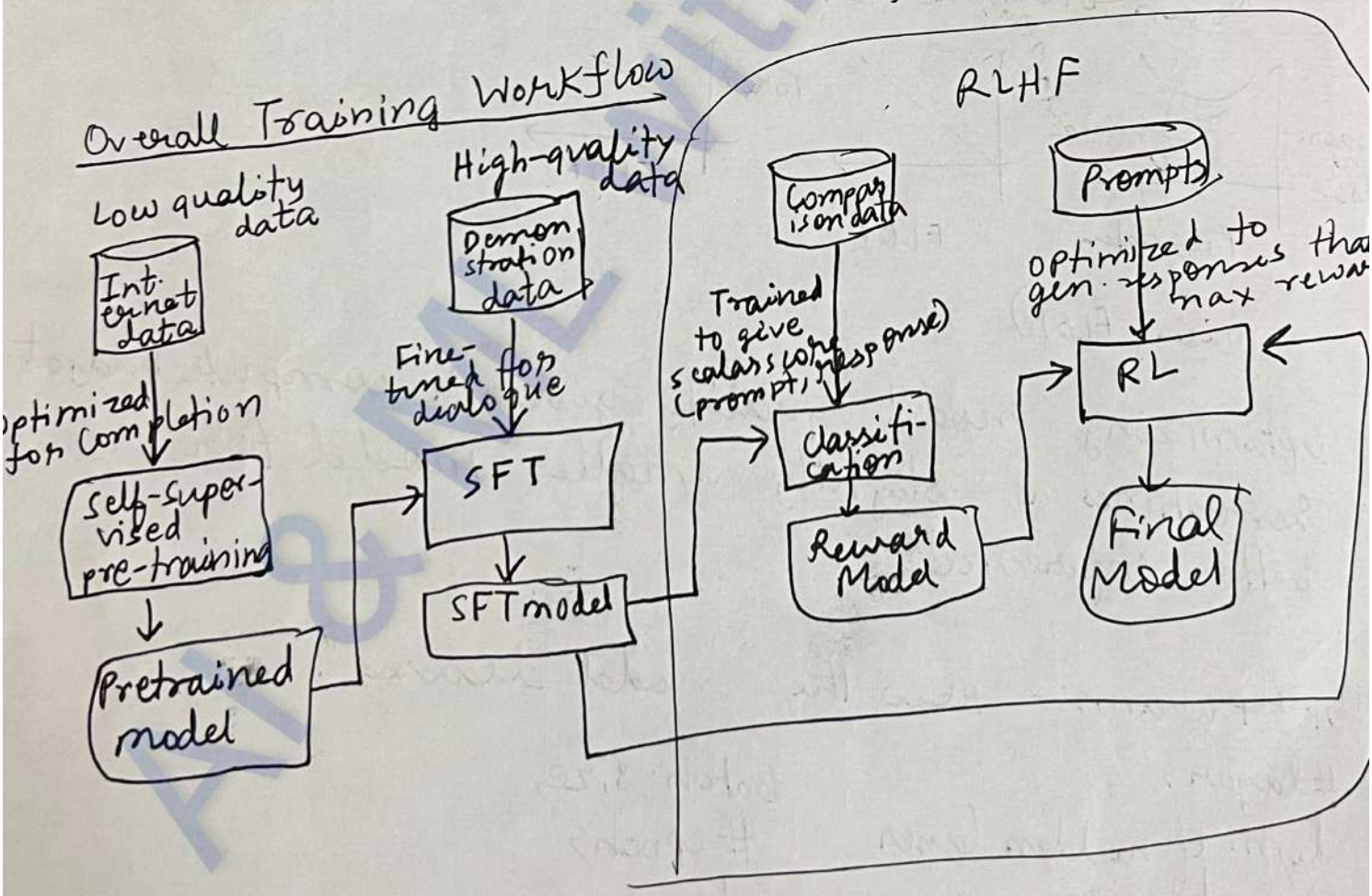
Scaling extrapolation :-
→ Hyperparameters for larger models?
→ Emergent abilities with large models, makes extrapolation less accurate.
→ Also no. of combinations.

Scaling bottlenecks :- Training data in internet grows slower than dataset size to train models.
→ AI-gen-data on Internet trains future models. (Forgets original data patterns).
→ Electricity consumption

3. Post Training

Pretrained: Text completion, not conversations
Indiscriminate biased ~~of~~ data from internet.

Overall Training Workflow



Aayush

Supervised finetuning:-

→ "How to make Pizza?"
→ (prompt, response) → demonstration data.
→ (High quality): InstructGPT.
→ labelers can be skewed (90% male).
→ Use heuristics to filter conversations
[A]: short para
[B]: "
[A]: "
[B]: "
pretrain + finetune yields better results.

Preference finetuning:- Imbibe human preference.

① Train a reward model that scores the foundation model o/Ps
② Optimize the foundation model to generate responses for which the reward model will give maximal score.

Reward model: Comparison data.
(prompt, win_response, lose_response), InstructGPT

r_θ:
(x, y_w, y_l)
$s_w = r(x, y_w)$: } scored by reward model
$s_l = r(x, y_l)$

Each (x, y_w, y_l)
loss $= \log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))$
θ to $\min[-E_x[\text{loss}]]$

Finetuning using the reward model:- Training done with PPO.

AI & ML with Ayush

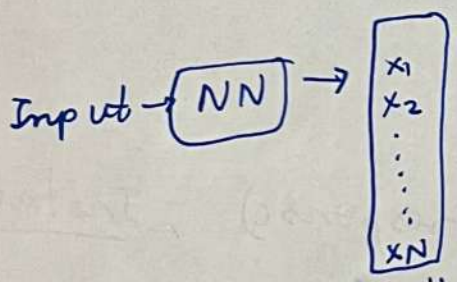
SFT & Preference finetuning :- Low quality pre-trained data

Some Companies skip RL
Just train reward model
Model gen. multiple O/Ps, and picks one which is given a high score by the reward model

sampling :- The way a model constructs it's O/Ps

Funda's :- To generate the next token, the LM first computes the prob. distribution over all tokens in the vocab.

→ Greedy sampling.
→ sample based on probability dist. over all possible



$$P_i = \text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

sampling strategies :- — either more creative
— more predictable

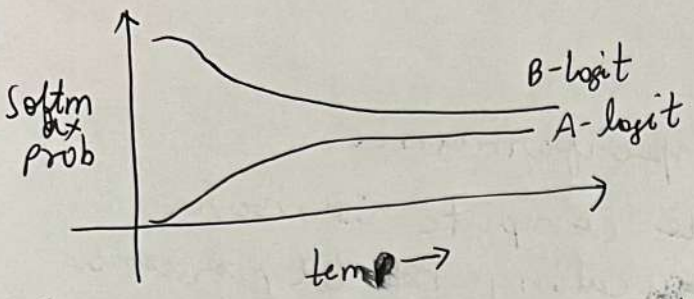
Temperature :- The color
High Temp ↑, prob. of rare tokens ↑

$\frac{x_i}{T}$

logits: [1, 2]
softmax: [0.27, 0.73]

temp = 0.5,
[0.12, 0.88]
B

A B
[1, 2]



Temp → 0
The model picks the largest logit.

Model providers return probability as logprobs
Avoids underflow problem.
prob. very small for many tokens (~100,000).
Exposing a model's log probs can be easier for others to replicate the model.

Top-K :- sampling strategy to reduce the computation Workload

$$\frac{e^{x_i}}{\sum_j e^{x_j}} \quad \forall v \in V$$

We pick TopK logits
softmax over topK logits.

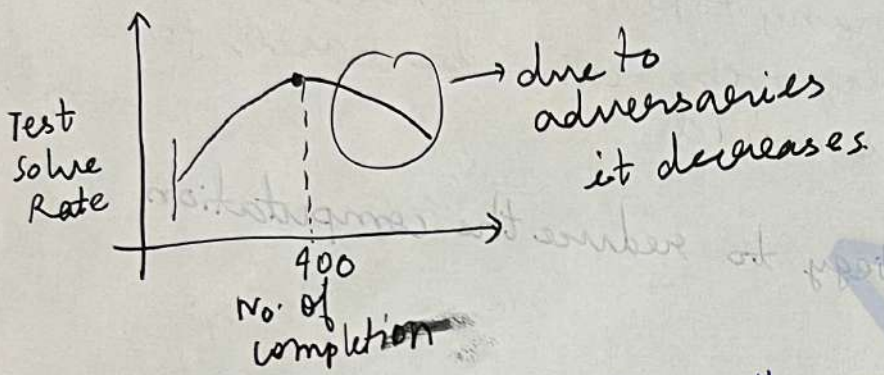
Top-P :- For more dynamic selection?
More contextually appropriate

Stopping Cond^n :- use stop words or <EOS>
downside is generating JSON

Test-time Compute :-
→ Discusses how a model might sample the whole O/P.
→ Improving a model's response quality.
→ Generate multiple responses, and use best of N technique.
→ We can use beam search.
→ Increase the diversity of O/Ps for more effective Test-time compute

AI & ML with Aayush

→ To diversify, we can use the sampling variables.
→ We work with log probs.
→ Average log probs.
→ use of verifiers improves performance.
→ Deep Mind: scale test time compute is more efficient than scaling model params.



Self-consistency: Picking the most common o/p among a set of o/Ps.

Structured Outputs:-

1. Tasks requiring structured o/Ps :- Semantic Parsing : Text-to-SQL.
2. Tasks whose o/Ps are used by downstream apps :- Agentic Workflows
   Model o/P → Tools.

→ Prompting: You can also use AI to validate the o/P of original prompt. (AI as a judge)

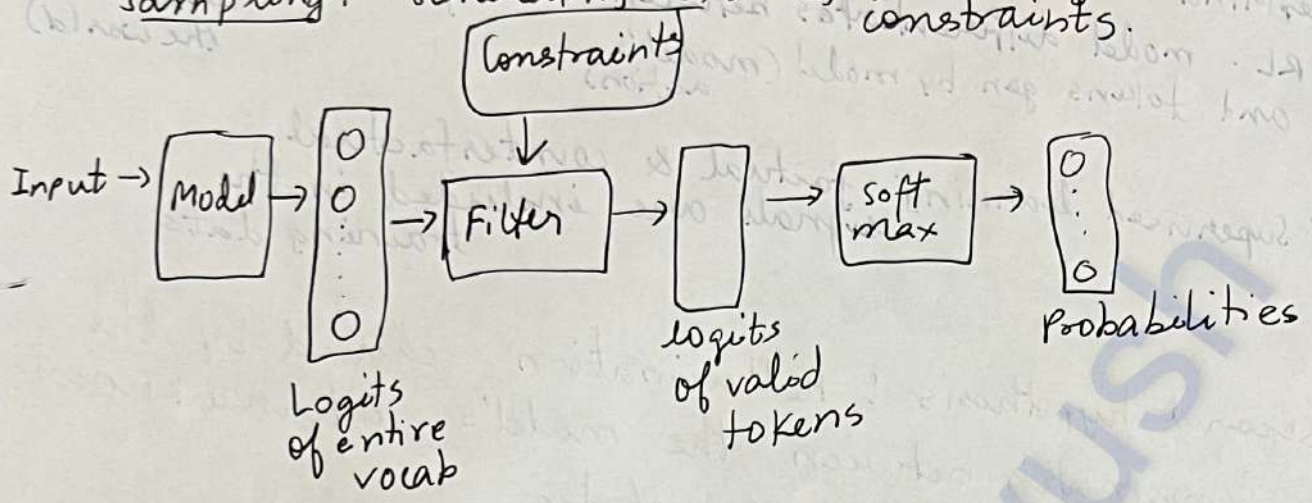
Two queries:
↳ To gen the o/P
↳ To validate the formnes

Can Incur more latency.

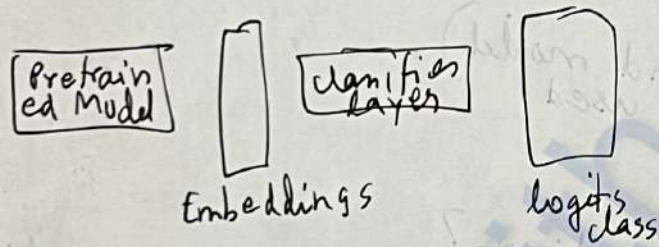
Aayush

Post-processing : manual correction.

Constrained sampling :- Generating text by following constraints.



Finetuning : For classification: Append a classifier head to the foundation model's architecture.



(Feature based transfer).

The Probabilistic Nature of AI :-

Inconsistency

The model generates very different responses for the same/slightly different prompts

To mitigate you need carefully crafted prompt & a memory system

Hallucination

Response not grounded in facts

Why?

↳ LMs can't differentiate between data that's given and what it generates.

Starting with a generated sequence slightly out of ordinary, model can expand upon it, and generate outrageously wrong facts.

self-delusion.

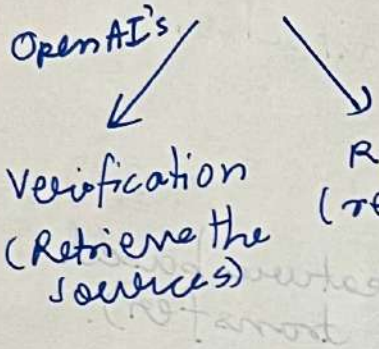
Mitigate?

DeepMind

→ RL. model differentiates between user provided prompts (obs, about the world) and tokens gen. by model (model's actions)

→ Supervised learning: factual & counterfactual signals are included in the training data

② Second hypothesis: Hallucination caused by the mismatch between the model's internal knowledge & labeller's knowledge



How to stop hallucinations?

We need to look into eval design., need a solid evaluation pipeline to detect failures and unexpected changes.

AI & ML with Aayush